

SETU Platform — Backend Codebase Audit & Shortcomings Report

Technical Gap Analysis, Requirement Reconciliation (PS 26002), & File-by-File Defect Log

Platform: SETU (Socio-Economic Transport & Utility Engine) | **Problem Statement:** SIH 26002 (MDoNER) | **Tech Stack:** Django 5.1, DRF, PostGIS / Spatial Compat, Scikit-Learn Risk Model | **Scope:** Backend & Matching Engine Codebase

1. Executive Summary & Assessment Overview

This audit report presents a rigorous examination of all backend services, spatial layers, API endpoints, and machine learning modules comprising the **SETU** platform. While the repository provides a solid architectural foundation that addresses the functional breadth of **SIH Problem Statement 26002** (MDoNER), critical gaps and shortcomings exist between the stated specifications in description.md and the live Python implementation. Key vulnerabilities include **disabled authorization on ViewSets** (AllowAny), a **critical resource inventory non-decrement bug** leading to phantom supply over-allocation, **synthetic straight-line routing** substituting for actual OSRM server computation, **N+1 SQL query bottlenecks** in command dashboard analytics (1,300+ queries per request), and the **absence of real-time SMS/Push gateway dispatchers**.

2. Problem Statement 26002 Requirement-by-Requirement Tally

Comparison of mandated requirements from the Ministry of Development of North Eastern Region (MDoNER) checklist (Items a–h) against the active backend implementation:

PS 26002 Requirement	Description Specification	Backend Implementation	Identified Shortcomings & Gaps	Status
(a) Road / Bridge Accessibility	Condition entity logging road status (blocked, flooded, landslide), GPS, district.	core/models.py core/views.py	Unauthenticated users can post fake road closures (AllowAny). No external automated sensor ingestion.	Partial
(b) Disruption Prediction (AI/ML)	Gradient Boosting Classifier evaluating slope, rain, soil, elevation, drainage.	matching_engine/risk_model/predict.py	Evaluates single coordinate point rather than road corridor polyline. Uses synthetic data and heuristic fallbacks for missing GIS layers.	Partial
(c) Alternate Routes & Delays	OSRM route GeoJSON generation; dynamic delay penalty; +45 min cascade.	core/services.py matching/services.py	No actual OSRM integration: generates synthetic 3-point straight line with arbitrary offset. Delay is hardcoded to static +45 min.	Deficient
(d) GPS Fleet Tracking	Vehicle tracking live GPS pings (/api/vehicles/{id}/ping/), status, and missions.	logistics/models.py logistics/views.py	No telemetry history log: overwrites current_location. No speed, heading, or breadcrumbs. Ping endpoint lacks operator auth check.	Partial
(e) Automated Hazard Alerts	Alert cascading engine broadcasting across SMS, Push, and In-App channels.	core/services.py (trigger_condition_alerts)	No SMS/Push gateway integration: only creates passive DB rows. No WebSockets for live push to dashboards.	Partial
(f) Geo-Tagged Photo Uploads	MediaAttachment entity for multipart photo/video evidence linked to Needs/Conditions.	core/models.py core/views.py	No file MIME/extension validation (security risk), no upload file size limit. Static media Docker routing path mismatch.	Substantial
(g) District Command Dashboard	District Bottleneck Index analytics endpoint (/api/dashboard/district-summary/).	dashboard/views.py	N+1 SQL query explosion: runs 11 SQL queries per district in a Python loop (1,320+ queries across 120 NER districts).	Slow
(h) Multilingual & Low-Net Sync	User language preferences (en, as, bn, hi), localized alert payloads, offline sync.	accounts/models.py core/models.py	Alert messages are hardcoded in English; no translation engine. No client UUID idempotency keys for offline retry handling.	Deficient

3. Exhaustive File-by-File Audit & Defect Log

Detailed analysis of every source file in the backend and matching engine:

3.1 Authentication & Authorization (backend/accounts/)

File: backend/accounts/permissions.py

Purpose: Defines role-based permission classes (IsFieldOfficerOrAdmin, IsTransportOperatorOrAdmin, IsDistrictAdmin, IsVerifiedProvider).

Identified Technical Problems:

- Shortcoming: Permission classes are fully defined but completely bypassed across the entire codebase because ViewSets override them with AllowAny.
- Shortcoming: Missing object-level permission hooks (e.g. ensuring a driver can only modify their assigned vehicle, or an NGO can only edit their own resources).

File: backend/accounts/views.py

Purpose: Provides endpoints for user registration, JWT login (CustomTokenObtainPairView), profile inspection (MeView), and user directory listing.

Identified Technical Problems:

- Shortcoming: RegisterView allows unrestricted role self-assignment — any caller can register themselves as 'district_admin' or 'admin' and automatically obtain is_staff privileges.
- Shortcoming: UserListView allows any authenticated user to view the full user directory, exposing phone numbers and assigned districts.

File: backend/accounts/serializers.py

Purpose: Handles user serialization, registration validation, and JWT token payload augmentation.

Identified Technical Problems:

- Shortcoming: Missing phone number format validation (E.164 regex) and district existence/state consistency checks.

3.2 Core Disaster & Resource Domain (backend/core/)

File: backend/core/views.py

Purpose: Exposes CRUD endpoints for Districts, Needs, Resources, Conditions, Allocations, Alerts, and AI Risk Prediction.

Identified Technical Problems:

- Critical Vulnerability: All ViewSets (NeedViewSet, ResourceViewSet, ConditionViewSet, AllocationViewSet, AlertViewSet) set `permission_classes = [AllowAny]`. Any unauthenticated caller can alter relief records or post fake emergency conditions.
- Crash Bug: In NeedViewSet, ResourceViewSet, and ConditionViewSet, unauthenticated creation triggers an unhandled IntegrityError (NOT NULL constraint failed on reported_by / provider) causing 500 Internal Server Errors.
- Security Deficit: `upload_attachment` actions accept any file without validating MIME type/extension or enforcing maximum file size limits (DoS vector).
- Security Deficit: AllocationViewSet allows any user to transition allocations to 'delivered', prematurely marking open relief needs as 'fulfilled'.

File: backend/core/services.py

Purpose: Contains `trigger_condition_alerts()` for event-driven hazard reactions and `generate_route_geojson()` for routing.

Identified Technical Problems:

- Critical Routing Gap: `generate_route_geojson()` does not call any OSRM server or routing engine. It generates a synthetic 3-point straight line with an arbitrary offset (+0.01 lat), which cannot represent real mountain road topography.

- **Heuristic Deficit:** Blockage reactions apply a flat hardcoded +45 min delay regardless of actual detour length or road segment classification.
- **Localization Gap:** Alert messages are hardcoded English strings ('CRITICAL: Road obstruction reported...'), ignoring the recipient's preferred_language.
- **Channel Incompleteness:** Alerts are only saved to the database with channel='app'. No external SMS (Twilio/Fast2SMS) or WebPush dispatches occur.

File: backend/core/models.py

Purpose: Defines District, Need, Resource, Condition, MediaAttachment, Allocation, and Alert.

Identified Technical Problems:

- **Data Model Deficit:** Allocation lacks an allocated_quantity field, preventing partial fulfillments or multi-depot relief splits.
- **Serialization Deficit:** District.boundary is a SpatialPolygonField, but DistrictSerializer lacks GeoJSON polygon formatting/validation logic.

File: backend/core/spatial_compat.py

Purpose: Fallback compatibility layer providing MockPoint and FallbackSpatialField when PostGIS/GDAL is unavailable.

Identified Technical Problems:

- **Performance Bottleneck:** FallbackSpatialField stores coordinates in JSONField. Without PostGIS spatial indexing (R-Tree/GiST), bounding-box spatial queries cannot run at the database level, forcing in-memory Python calculations.

3.3 Matching Engine & Inventory Orchestration (backend/matching/ & matching_engine/)

File: backend/matching/services.py

Purpose: Orchestrates resource discovery, candidate scoring via matching_engine, and allocation confirmation.

Identified Technical Problems:

- **Critical Inventory Bug:** confirm_match_and_allocate() marks Need as 'matched' and creates an Allocation, but NEVER decrements Resource.quantity_available. Multiple needs can allocate the same stockpile simultaneously (phantom inventory).
- **Concurrency Deficit:** Does not use select_for_update() on candidate resources, risking race conditions and duplicate allocations during concurrent dispatches.
- **Hazard Scoring Blindspot:** Corridor hazard evaluation only inspects the candidate depot's home district (cand.district). If the transport corridor traverses intermediate blocked districts, the hazard is completely ignored.

File: matching_engine/scoring.py

Purpose: Multi-criteria weighted scoring algorithm evaluating Urgency, Proximity (exponential decay), Verification, Quantity Fit, and Delay Risk.

Identified Technical Problems:

- **Logic Deficit:** Proximity decay assumes a fixed 25 km half-life without adjusting for mountainous terrain non-linear travel times.
- **Logic Deficit:** Quantity fit score penalizes partial fulfillment linearly without multi-depot bundling options.

File: matching_engine/risk_model/predict.py & train.py

Purpose: GradientBoostingClassifier pipeline trained to predict disruption probability based on geo-climatic indicators.

Identified Technical Problems:

- **Point vs. Corridor Deficit:** predict_risk() only evaluates a single coordinate pair rather than an entire route polyline.
- **Synthetic Data Deficit:** Model is trained on synthetic approximations. When missing features (elevation, soil, drainage), it relies on hardcoded heuristics rather than querying live GIS/DEM rasters.

3.4 Logistics, Telemetry & Analytics (backend/logistics/ & backend/dashboard/)

File: backend/logistics/models.py & views.py

Purpose: Vehicle fleet registry and real-time GPS coordinate ingestion endpoint (/api/vehicles/{id}/ping/).

Identified Technical Problems:

- **Telemetry Gap:** Vehicle only stores current_location and last_ping_at. There is no telemetry history/breadcrumb log to reconstruct historical routes or calculate off-route deviations.
- **Security Deficit:** VehicleViewSet.ping() allows any caller to update any vehicle's location without verifying that the authenticated user is the assigned operator.
- **Validation Deficit:** VehiclePingSerializer lacks coordinate bounding box checks (allowing invalid lat/lon values).

File: backend/dashboard/views.py

Purpose: DistrictSummaryView (/api/dashboard/district-summary/) calculating District Bottleneck Indices and regional health.

Identified Technical Problems:

- **Severe Performance Issue (N+1 Queries):** Iterates over all districts in Python and executes 11 distinct SQL queries per district. For 120 NER districts, this fires 1,320+ SQL queries per GET request.
- **Remediation Needed:** Must refactor to use Django ORM annotations (.annotate(Count('needs', filter=...))) into a single aggregated SQL query.

3.5 Infrastructure, Configuration & DevOps (backend/config/, Docker, Nginx)

File: docker-compose.yml & backend/Dockerfile

Purpose: Production container orchestration declaring PostGIS, Unicorn Django backend, Frontend, and Nginx reverse proxy.

Identified Technical Problems:

- **Missing Command:** Backend container does not run 'python manage.py collectstatic --noinput' on startup.
- **Path Mismatch:** Backend sets MEDIA_ROOT=/app/backend/media, while infra/nginx.conf maps /media/ to /app/media/, causing media file 404s under Nginx.

File: backend/config/settings.py

Purpose: Django central settings (REST framework, JWT, database connection, CORS).

Identified Technical Problems:

- **Missing Rate Limiting:** REST_FRAMEWORK lacks DEFAULT_THROTTLE_CLASSES / DEFAULT_THROTTLE_RATES.
- **Missing WebSockets:** Settings and ASGI configuration lack Django Channels routing and Redis channel layers for real-time live pushing.

4. Core Architecture Deficits Deep-Dive

Categorization of the systemic architectural shortcomings discovered during the audit:

1. Access Control & Security Posture

- Public Read/Write on All Endpoints: Overriding ViewSet permissions with AllowAny exposes critical disaster infrastructure to malicious tampering.
- Unauthenticated 500 Server Crashes: Serializers assume authenticated users and fail with raw IntegrityError exceptions when unauthenticated requests are received.
- File Upload Exploitation: Unrestricted multipart upload endpoints allow arbitrary file extensions and unbounded upload sizes.

2. Inventory Integrity & Transactional Consistency

- Phantom Inventory Bug: Confirming an allocation does not decrement available stock, allowing double-allocation of finite medical/food supplies.
- Missing Concurrency Protection: Lack of row-level locks (select_for_update) exposes concurrent dispatch operations to race conditions.
- Single-Depot Constraint: Allocation model lacks support for partial fulfillments and multi-source supply aggregation.

3. AI Disruption Modeling & Routing Reality

- Mock OSRM Geometry: The route generator produces a synthetic 3-point straight line with an arbitrary offset rather than real topological road geometries.
- Intermediate Transit Blindspots: Hazard scoring inspects only the origin district and misses active landslides in intermediate transit corridors (e.g. NH-6).
- Single-Point Inference: AI risk prediction evaluates single coordinates rather than continuous road corridors.

4. Real-Time Telemetry & Communication Gaps

- Volatile Telemetry: Overwriting vehicle coordinates without a historical breadcrumb log prevents route reconstruction and off-route detection.
- Passive Alerting: Alerts exist only as database records; no external SMS, WebPush, or WebSocket push notifications are dispatched.
- Unlocalized Alerting: Generated alert strings are hardcoded in English, ignoring Assamese, Bengali, and Hindi user preferences.

5. Scalability & Database Performance

- Dashboard N+1 Query Explosion: 11 SQL queries executed per district in a Python loop creates severe latency on regional dashboards.
- Spatial Indexing in Fallback Mode: JSON-based coordinate storage prevents native spatial indexing when running in fallback SQLite environments.

5. Prioritized Remediation Roadmap

Priority	Action Item	Target File(s)	Expected Impact
P0 (Immediate)	Enforce RBAC Permissions: Replace AllowAny with IsAuthenticated and specific role permissions across all ViewSets. Add user fallback checks.	core/views.py matching/views.py logistics/views.py	Prevents unauthorized tampering and resolves unhandled 500 server crash exceptions.
P0 (Immediate)	Fix Resource Inventory Bug: Wrap confirmation in select_for_update() and decrement quantity_available. Add allocated_quantity to Allocation.	matching/services.py core/models.py	Eliminates phantom inventory double-allocations and ensures stock consistency.
P1 (High)	Optimize Dashboard Analytics: Replace Python loop queries with single aggregated ORM query using .annotate() and Count(filter=Q(...)).	dashboard/views.py	Reduces database queries from 1,320+ to 1, providing sub-50ms dashboard response times.
P1 (High)	Integrate Real Routing Engine: Connect actual OSRM / OpenRouteService client for authentic driving geometry and corridor hazard intersections.	core/services.py matching/services.py	Enables realistic mountain travel times, detour delay calculations, and cross-district hazard detection.
P1 (High)	Implement Telemetry Breadcrumb History: Create VehicleLocationPing model to store full GPS history, speed, and heading.	logistics/models.py logistics/views.py	Enables live breadcrumbs on maps and off-route deviation tracking for relief fleets.
P2 (Medium)	Multilingual Alerting & Notification Gateways: Implement translation dictionaries (as, bn, hi) and integrate external SMS/WebPush dispatchers.	core/services.py config/settings.py	Delivers localized disaster notifications across SMS, Push, and Web channels.
P2 (Medium)	DevOps & File Upload Security: Fix Nginx media routing paths, add collectstatic to Docker startup, and add MIME/size validation on uploads.	docker-compose.yml infra/nginx.conf core/serializers.py	Ensures smooth production asset serving and prevents file upload exploits.